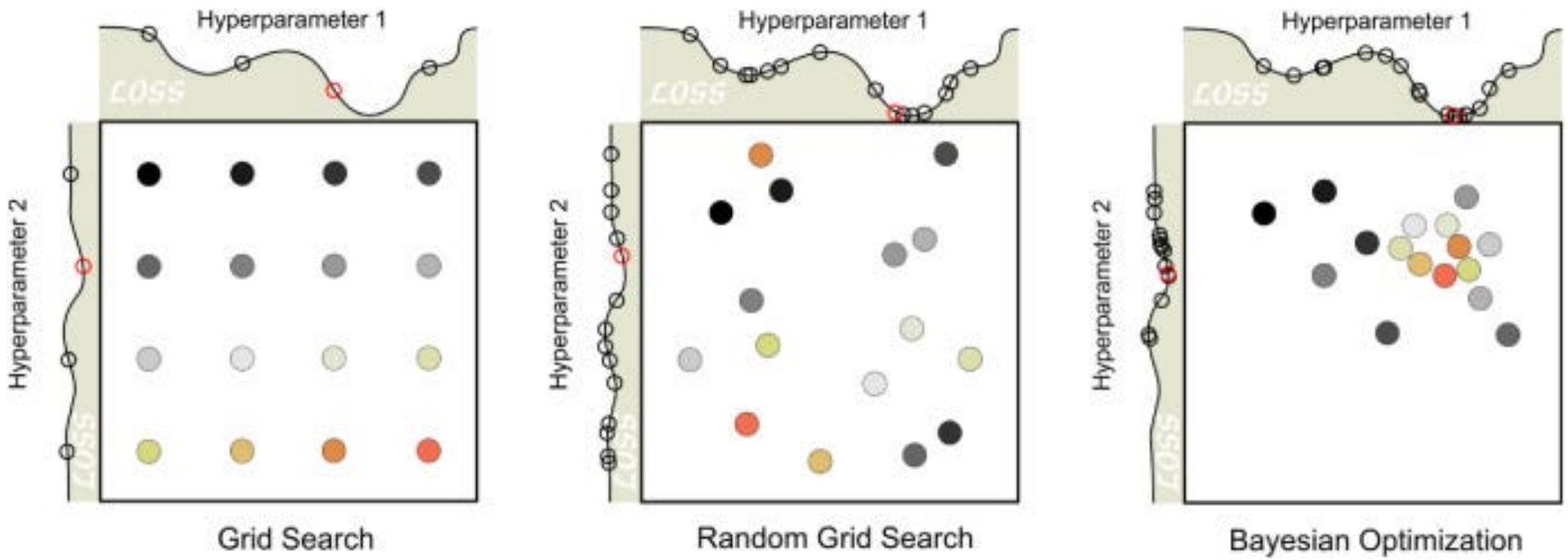
AutoML, Autoresearch, MLOps +@

26.4.8

서민교

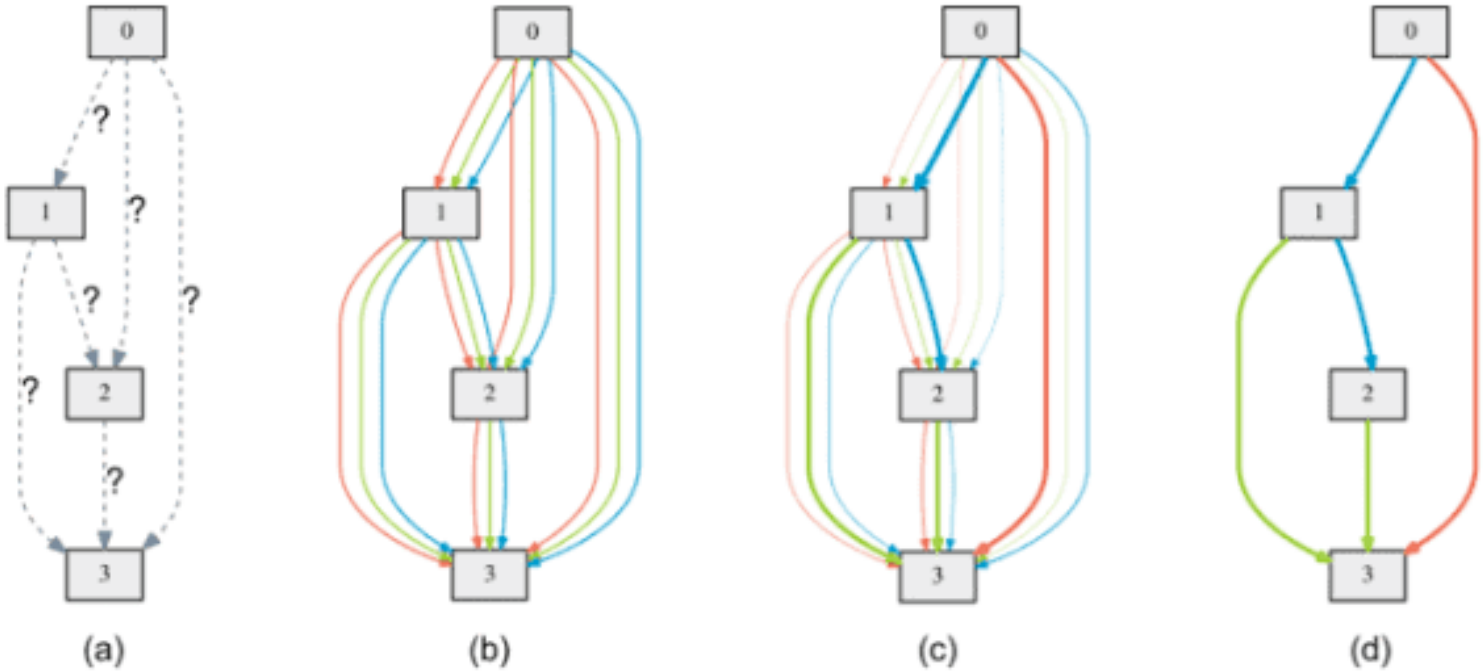
1. AutoML이란 무엇인가?

- 모델 개발 탐색 일부 자동화
- 대표 대상: `model selection`, `hyperparameter tuning`, `pipeline search`
- 핵심: 비교적 주어진 `search space` 안 최적 설정 탐색



2. `Neural Architecture Search`는 AutoML의 확장이다

- parameter 대신 architecture 탐색
- AutoML의 `더 넓은 search space` 확장선
- 그래도 중심은 여전히 모델/파이프라인 후보 탐색



`hyperparameter search → pipeline search → architecture search`

3. `Autoresearch`는 어떻게 등장했고 무엇이 다른가?

- karpathy/autoresearch: 작은 training setup 위 `read → edit → run → keep-or-revert` loop 제시
- 연구 workflow 일부를 agent가 직접 수행하며, 설정 탐색을 넘어 `code`, `module`, `experiment` 자체 수정
- AutoML의 `fixed search space` 바깥으로 확장
- 이후 RD-Agent, AI-Scientist, GPT Researcher 등으로 빠르게 확장

4. Agent 작업 흐름

- 코드 읽기, baseline 파악
- 작은 가설 하나 선택
- 학습 코드나 설정 수정
- 짧은 실험 실행, metric 확인
- 나쁘면 revert, 의미 있으면 keep
- 핵심: `edit 한 번`이 아니라 `짧은 실험 loop의 누적`

```
Question → Read → Edit → Run → Analyze → Next experiment
```

5. AutoML vs. Autoresearch

항목	AutoML	Autoresearch
탐색 대상	config, pipeline, architecture	hypothesis, code, module, experiment
핵심 질문	어떤 설정이 가장 좋은가	다음에 어떤 실험을 해야 하는가
edit 단위	parameter / architecture	code / module / pipeline / experiment
평가 방식	objective 중심	objective + reasoning + iteration
위험	비효율적 탐색	incoherent search, metric hacking
필요한 인프라	experiment infra	experiment + memory + harness

6. 사용례와 확장

사용례

- 문헌 조사 / deep research: GPT Researcher
- 코드 수정 + 실험 반복: karpathy/autoresearch, RD-Agent
- end-to-end 연구 자동화: AI-Scientist

확장

- benchmark / evaluation: MLE-bench, MLAgentBench, MLR-Bench
- plugin / skill 생태계: awesome-autoresearch, Awesome Auto Research Tools
- memory, reusable modules, hardware fork

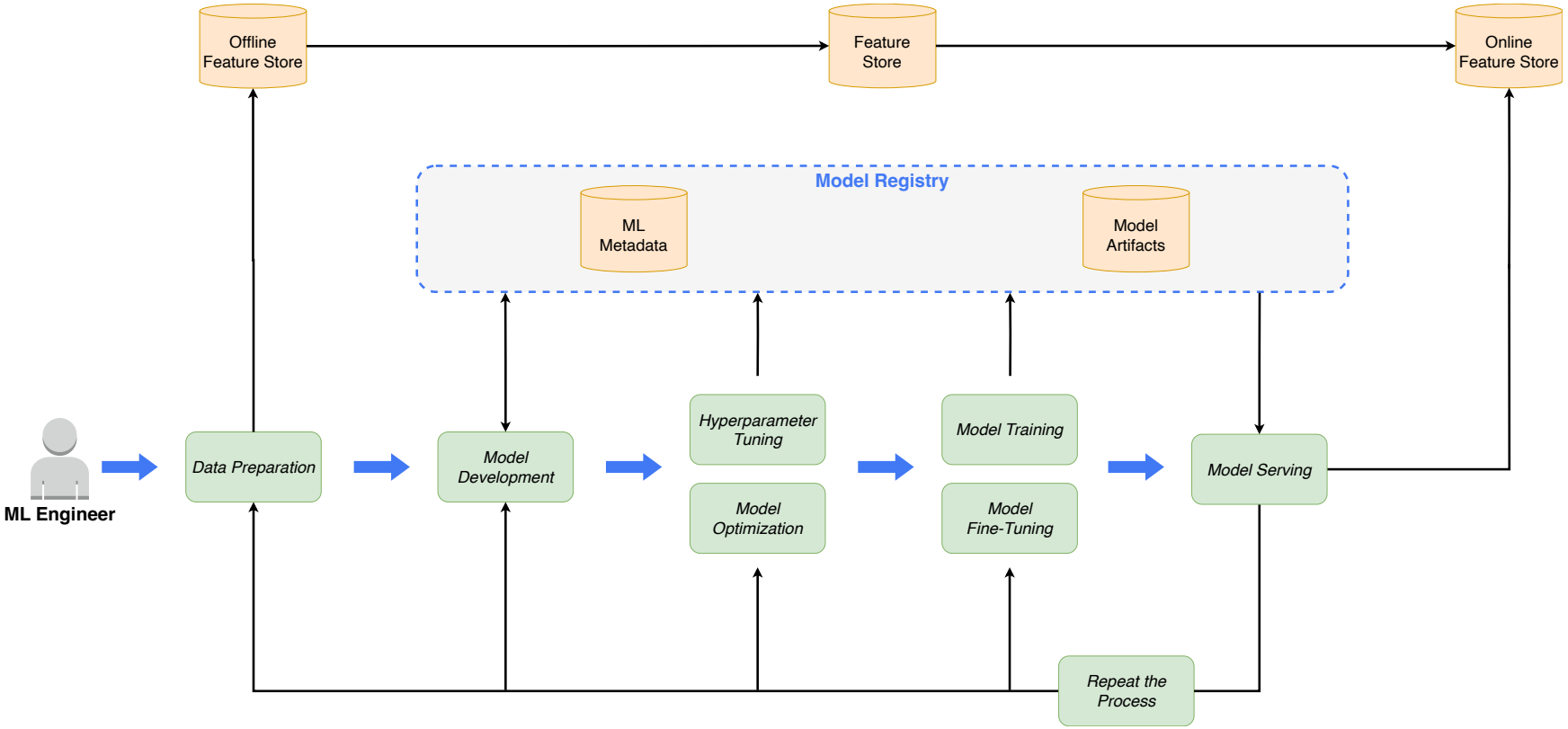
7. 체계적인 실험 관리의 필요

- 공통 문제: `많은 run` 비교와 누적
- 필수 요소: `tracking`, `lineage`, `orchestration`
- agent edit가 들어오면 `artifact`, `promotion`, `monitoring`, `cost control` 중요도 상승
- 결국 운영 문제

8. AutoML과 Autoresearch가 공통으로 요구하는 MLOps 요소

요소	AutoML에서의 역할	Autoresearch에서의 역할
tracking	sweep 비교	hypothesis / code edit history 비교
orchestration	search job 실행	agent + eval job 실행
registry / lineage	best model 승격	experiment / prompt / code provenance 보존
monitoring / cost	retrain trigger, SLO	budget, drift, unsafe promotion guardrail

9. MLOps는 모델 개발, 관리, 배포 파이프라인을 유지 관리하는 작업이다



- Autoresearch loop는 이 큰 ML lifecycle 안의 일부
- 실제 시스템: `data`, `experiment`, `model registry`, `deployment`, `monitoring`
- 핵심 역할: `지속 운영`, `추적`, `승격`, `유지관리`

10. 두 질문

- 성능: `(AutoML의 도메인에서)` `Autoresearch`는 baseline보다 낫나
- 과정: `Autoresearch` 과정에 어떤 harness가 있으면 좋을까
- 질문 1은 `solver` 비교다
- 질문 2는 `process` 비교다

11. 질문 1 셋업

- baseline: `Optuna TPE`, `SMAC3`
- agent: `Autoresearch`
- hybrid: `agent + TPE`, `agent + SMAC3`
- 비교 조건: 같은 search space, benchmark, budget

12. Optuna TPE

- 핵심 아이디어: 좋은 trial과 나머지 trial을 분리해 density를 학습한다
- 다음 제안은 `good / rest` 비율이 큰 쪽을 우선 샘플링한다
- 강점: bounded search space에서 incumbent 근처를 빠르게 좁힌다
- 성격: `exploit-heavy` baseline이라 agent 없는 강한 비교선이 된다

13. SMAC3

- 핵심 아이디어: surrogate model로 promising region을 예측한다
- evaluation budget은 racing으로 유망한 config에 더 배분한다
- 강점: categorical, mixed, conditional search space 대응이 강하다
- 성격: `model-based search` baseline이라 TPE와 다른 편향을 준다

14. 결과 TBD

TBD

- 실험 진행 중
- 동일 조건 결과 표 추가 예정
- `best val`, `best hidden`, search trace를 함께 정리할 예정
- 질문 1의 결론은 여기서 닫힌다

15. 질문 2 셋업

- 비교 대상은 agent profile 세 가지다
- `Ratchet` : incumbent 근처 exploit
- `Screening` : main effect 분리
- `Advanced` : interaction과 refinement까지 포함한 staged DOE

16. 왜 Harness

- 없으면: attribution 약화, 단계 혼합
- 결과: 잘 정리된 random search로 퇴화
- 큰 수정과 작은 튜닝이 같은 레벨로 섞인다
- 검증 실험이 뒤로 밀리기 쉽다

17. DoE Harness

- 순서: `screening → interaction → refinement`
- 장점: effect attribution, 다음 라운드 설계
- 강점: 무엇을 바꿨는지보다 무엇이 먹혔는지 읽기 쉽다
- 목적: 최고점 하나보다 `실험 질서` 를 만든다

18. 세 프로파일

Profile	역할	초점
`Ratchet`	incumbent 근처 exploit	빠른 개선
`Screening`	main effect 분리	요인 선별
`Advanced`	interaction / refinement	순차 설계

- 같은 search space에서도 운영 정책은 다르다

19. 실험 설정

항목	설정
benchmark	`cifar10_real`
model	`mlp`
conditions	`14`
budget	condition당 `100 + finalize 1`

- 비교 대상: `plain`, `TPE`, `SMAC`, `TPE+SMAC`, direct
- 목표: code edit가 아니라 bounded `AutoML + harness` 비교

20. 탐색 축

그룹	변수
preprocessing	`normalization`, `projection`, `resampling`
architecture	`hidden_dims`, `activation`, `norm layer`
optimization	`solver`, `lr`, `batch`
regularization	`wd`, `dropout`, `noise`

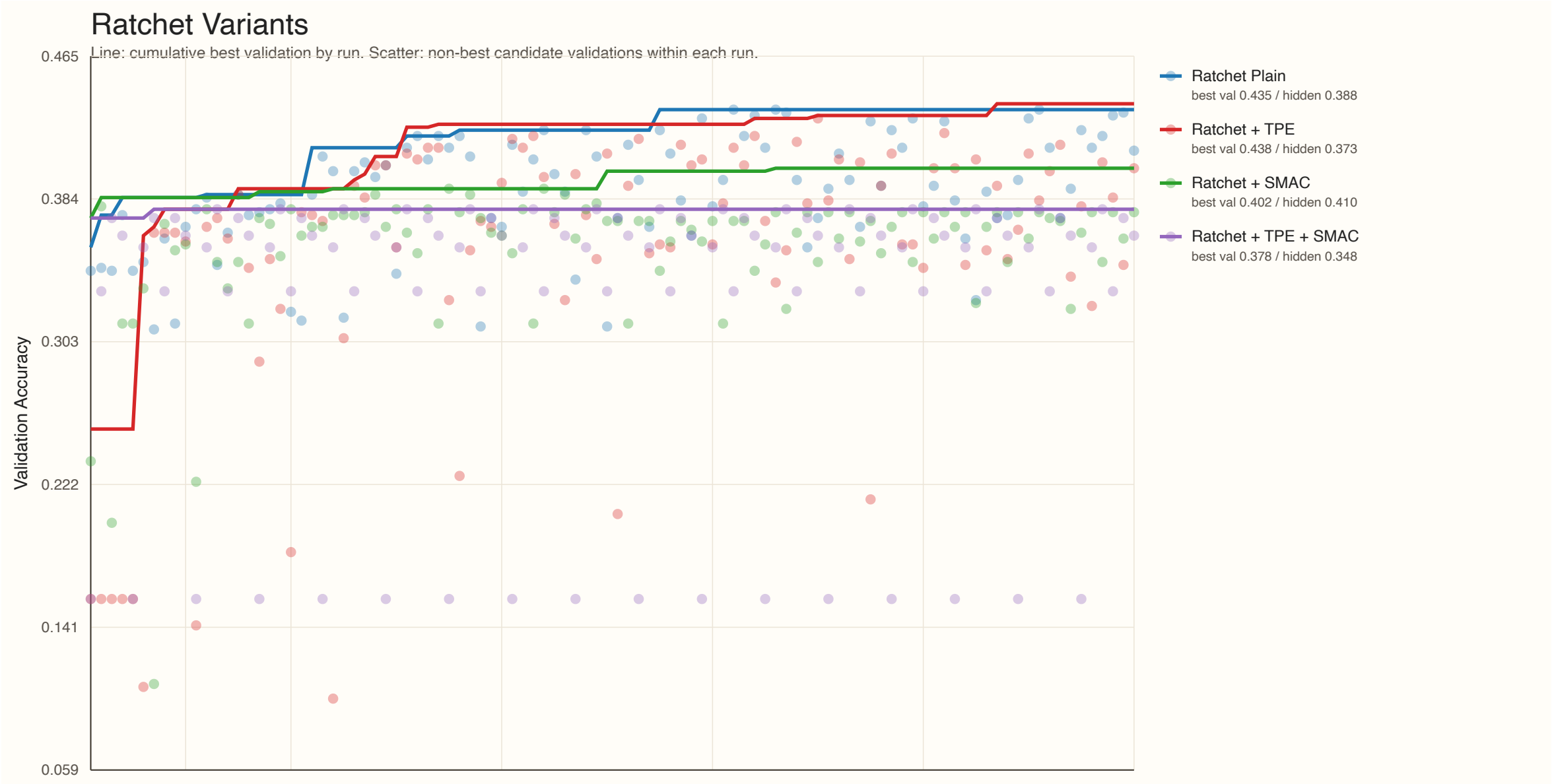
- 한 축만 바꾸는 round와 여러 축을 묶는 round를 구분해서 본다

21. 결과 요약

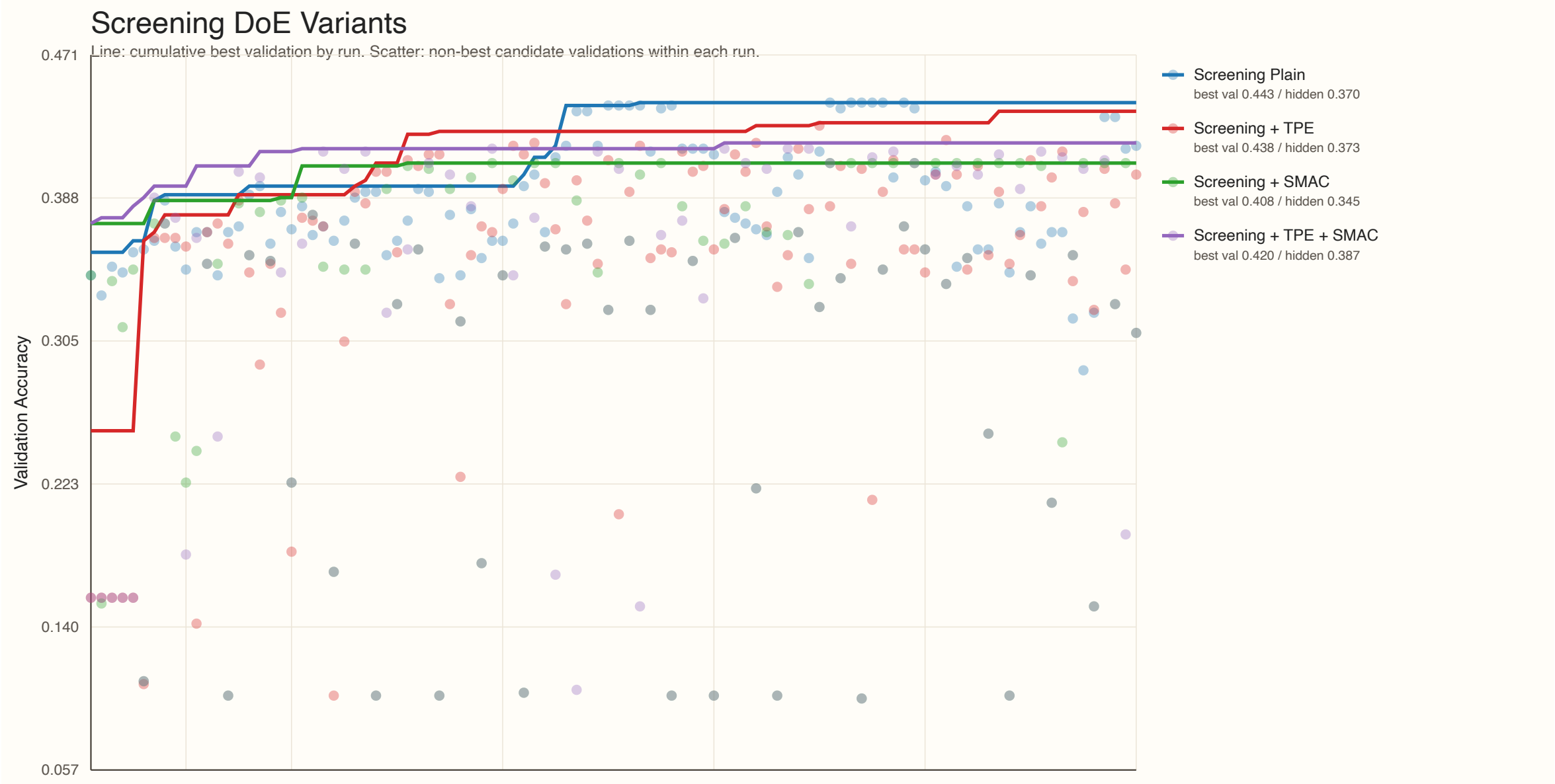
관점	조건	점수
best val	`screening_plain`	`0.4433`
best hidden	`ratchet_smac`	`0.4100`
best direct hidden	`tpe_direct`	`0.3733`

- 핵심: validation winner와 finalize winner가 다르다.
- 함의: harness가 peak와 generalization을 다르게 만든다.
- 따라서 mid-run 최고점만으로 harness를 평가하면 오판한다.

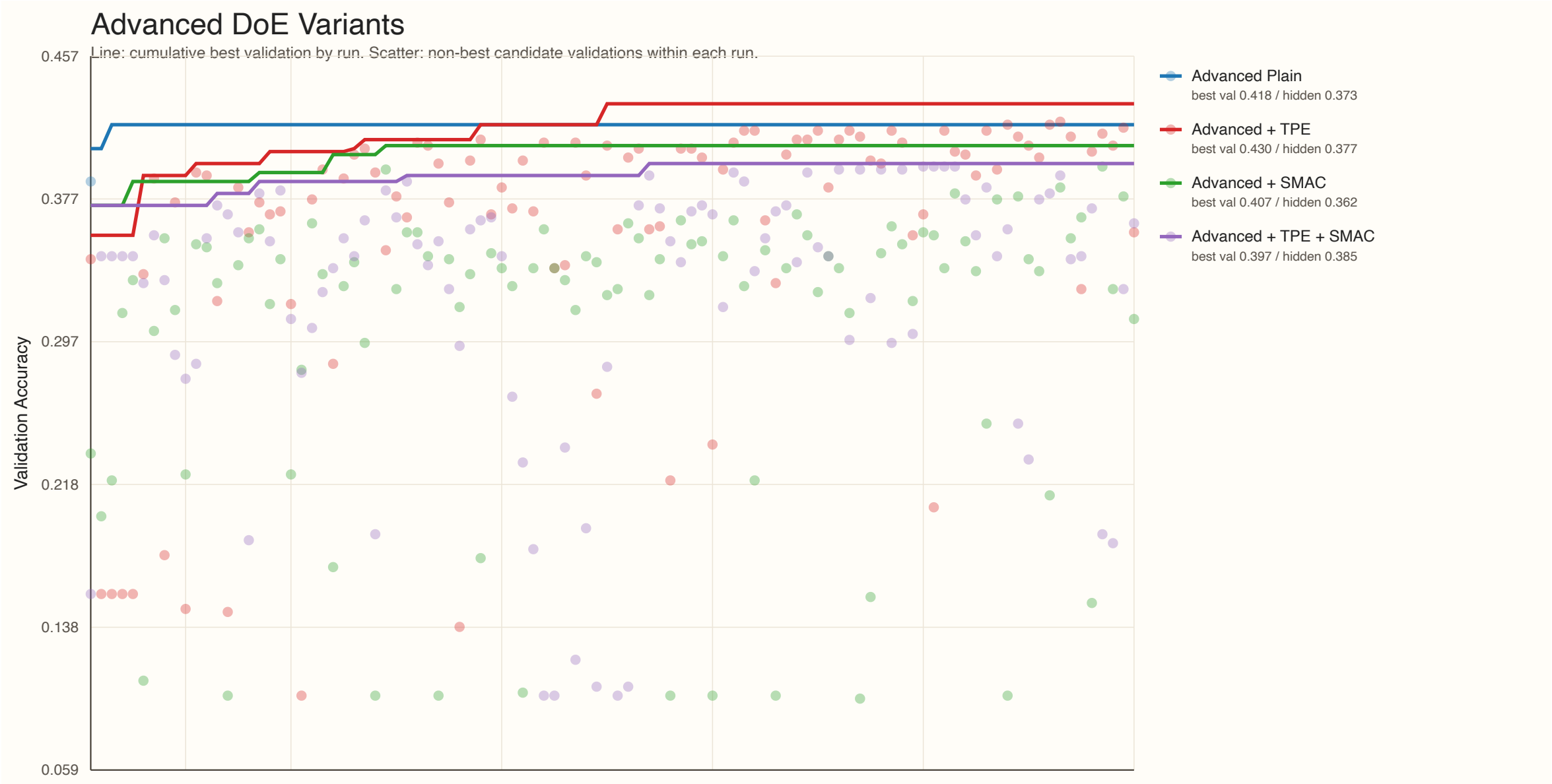
22. Ratchet


Ratchet Variants
Line: cumulative best validation by run. Scatter: non-best candidate validations within each run.
Validation Accuracy
0.465
0.384
0.303
0.222
0.141
0.059
Ratchet Plain
best val 0.435 / hidden 0.388
Ratchet + TPE
best val 0.438 / hidden 0.373
Ratchet + SMAC
best val 0.402 / hidden 0.410
Ratchet + TPE + SMAC
best val 0.378 / hidden 0.348

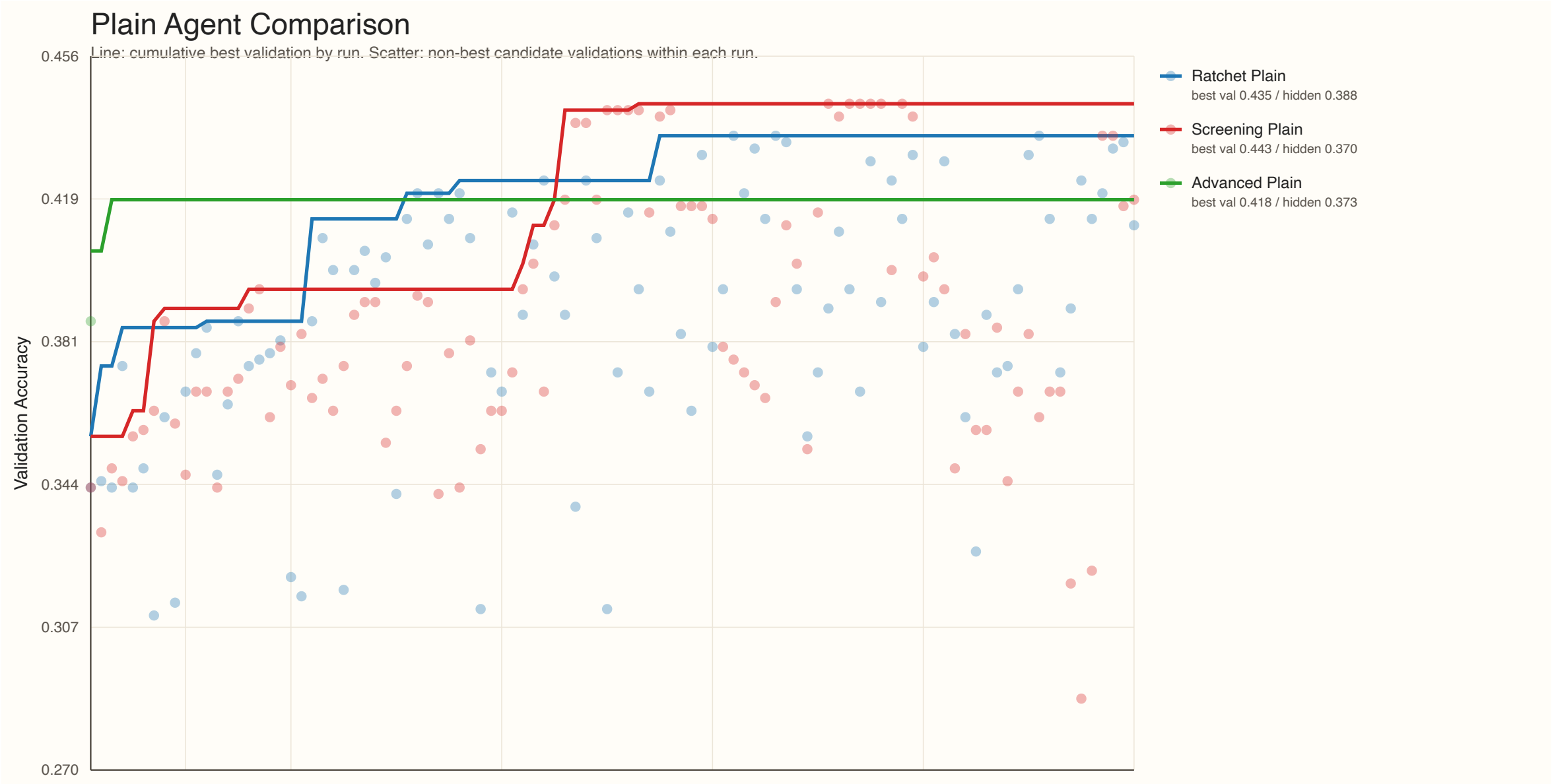
23. Screening



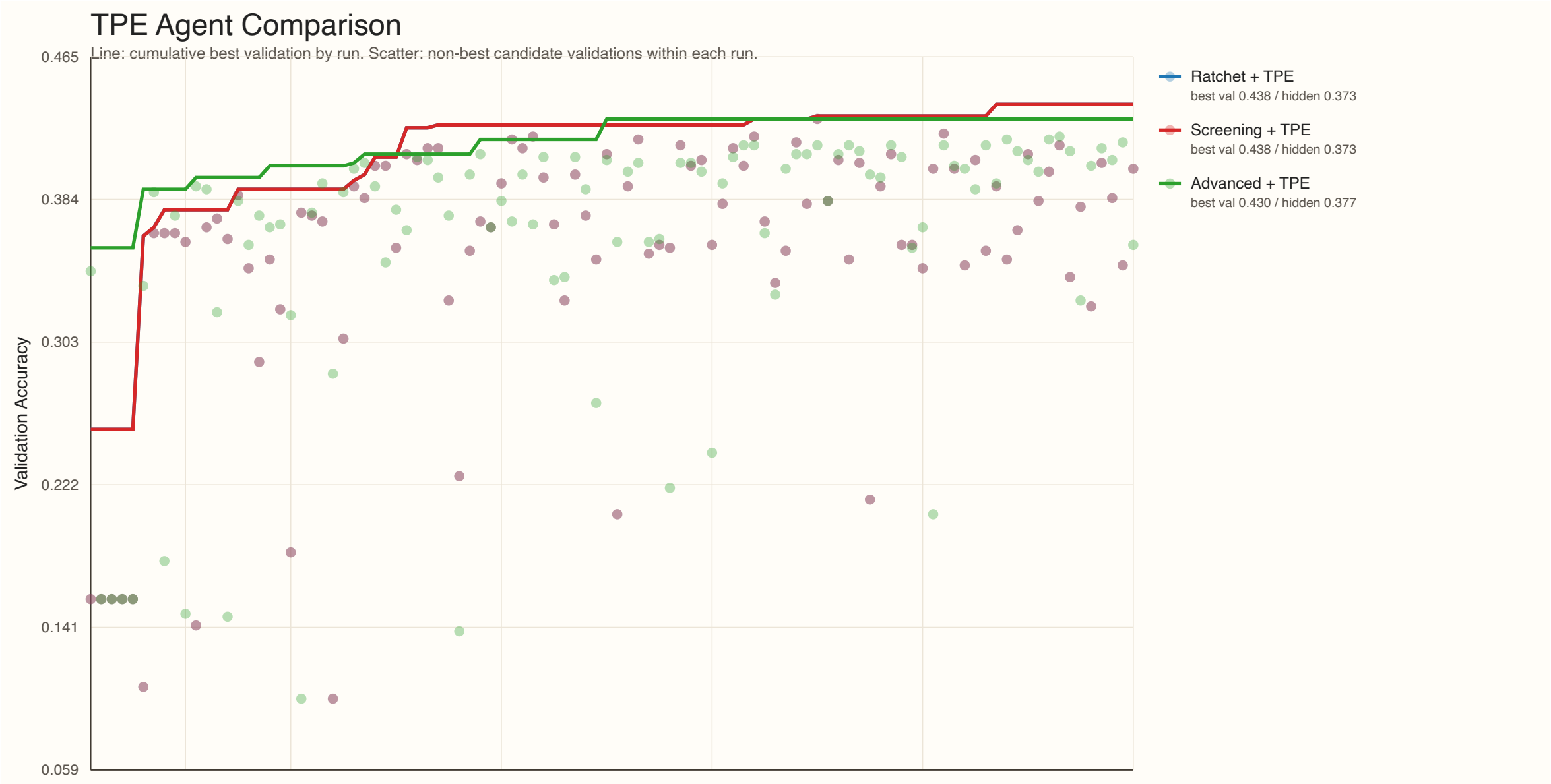
24. Advanced


Advanced DoE Variants
Line: cumulative best validation by run. Scatter: non-best candidate validations within each run.
Validation Accuracy
0.457
0.377
0.297
0.218
0.138
0.059
Advanced Plain
best val 0.418 / hidden 0.373
Advanced + TPE
best val 0.430 / hidden 0.377
Advanced + SMAC
best val 0.407 / hidden 0.362
Advanced + TPE + SMAC
best val 0.397 / hidden 0.385

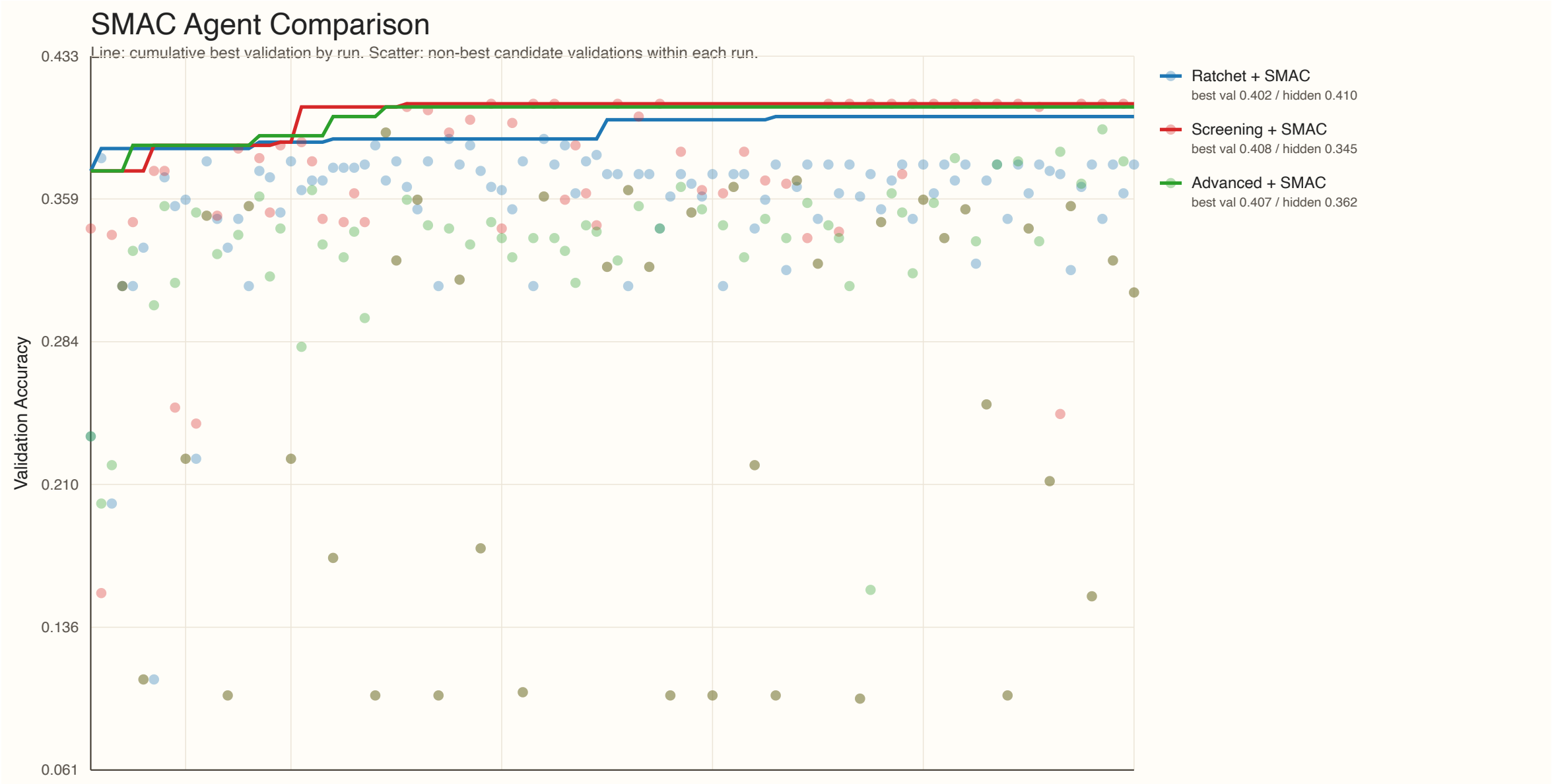
25. Plain


Plain Agent Comparison
Line: cumulative best validation by run. Scatter: non-best candidate validations within each run.
Validation Accuracy
0.456
0.419
0.381
0.344
0.307
0.270
Ratchet Plain
best val 0.435 / hidden 0.388
Screening Plain
best val 0.443 / hidden 0.370
Advanced Plain
best val 0.418 / hidden 0.373

26. TPE


TPE Agent Comparison
Line: cumulative best validation by run. Scatter: non-best candidate validations within each run.
Validation Accuracy
0.465
0.384
0.303
0.222
0.141
0.059
Ratchet + TPE
best val 0.438 / hidden 0.373
Screening + TPE
best val 0.438 / hidden 0.373
Advanced + TPE
best val 0.430 / hidden 0.377

27. SMAC


SMAC Agent Comparison
Line: cumulative best validation by run. Scatter: non-best candidate validations within each run.
Validation Accuracy
0.433
0.359
0.284
0.210
0.136
0.061
Ratchet + SMAC
best val 0.402 / hidden 0.410
Screening + SMAC
best val 0.408 / hidden 0.345
Advanced + SMAC
best val 0.407 / hidden 0.362

28. 핵심 함의

- harness effect는 advisor effect만큼 크다.
- validation 최적화와 finalize 최적화는 다른 문제다.
- dual advisor는 항상 단일 advisor를 이기지 않는다.
- 따라서 harness 평가는 `누가 더 똑똑한가` 보다 `무엇을 어떻게 보게 만드는가` 를 봐야 한다.

29. 운영 교훈

- 지표: `best val`만 보지 말고 `finalize`, `artifact completeness`를 같이 봐야 한다.
- 설계: `isolate / history / finalize` 분리가 있어야 중간 최고와 최종 승자를 함께 읽는다.
- 로그: advisor trace에는 invalid proposal이 섞일 수 있어 guard가 필요하다.
- 판정: run 수와 history row 수를 혼동하면 early finalize가 생긴다.

30. 한계

- 단일 benchmark, single split
- budget `100`, top-k reseeding 없음
- code-edit autoresearch는 아직 제외
- 따라서 이번 결론은 `bounded AutoML harness` 범위에 한정된다

31. References

구분	예시
curated landscape	awesome-autoresearch, Awesome Auto Research Tools
end-to-end systems	karpathy/autoresearch, RD-Agent, AI-Scientist
deep research	GPT Researcher
evaluation	MLE-bench, MLAgentBench, MLR-Bench
optimization baselines	Optuna docs, SMAC3 docs
visuals	AutoML image, NAS image, Kubeflow model registry lifecycle image